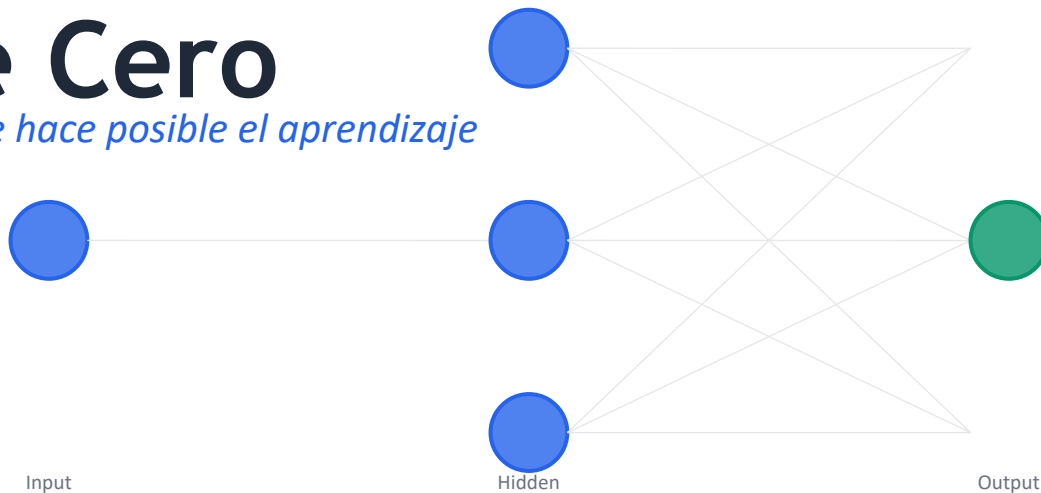


Backpropagation desde Cero

El algoritmo que hace posible el aprendizaje



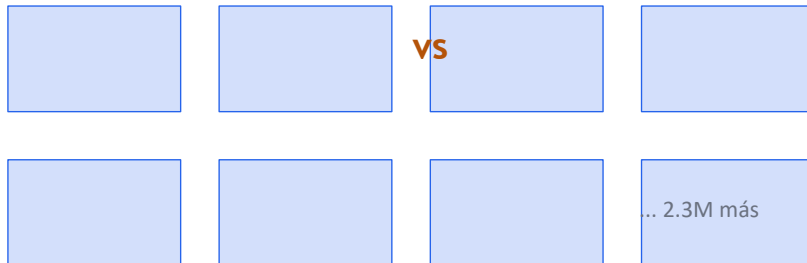
El Problema



2,359,296

pesos en 1 capa

Probar 1 peso = 1 forward pass

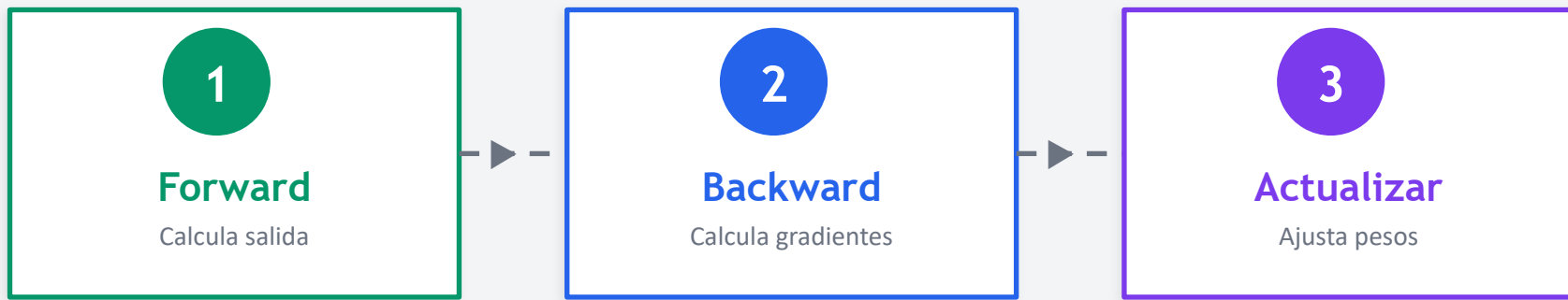


Backpropagation calcula TODOS los gradientes en UN solo backward pass

Pausa y piensa

¿Cuánto tardarías en probar 2.3M de pesos uno por uno?

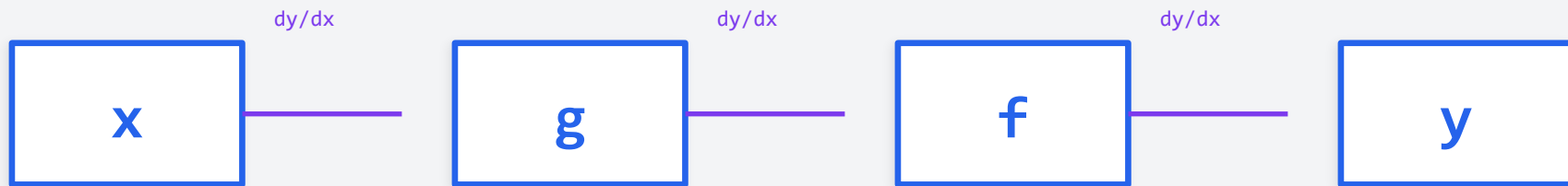
La Solución



Pausa y piensa

Detective: observa (forward) y reconstruye (backward) para hallar al culpable

Regla de la Cadena



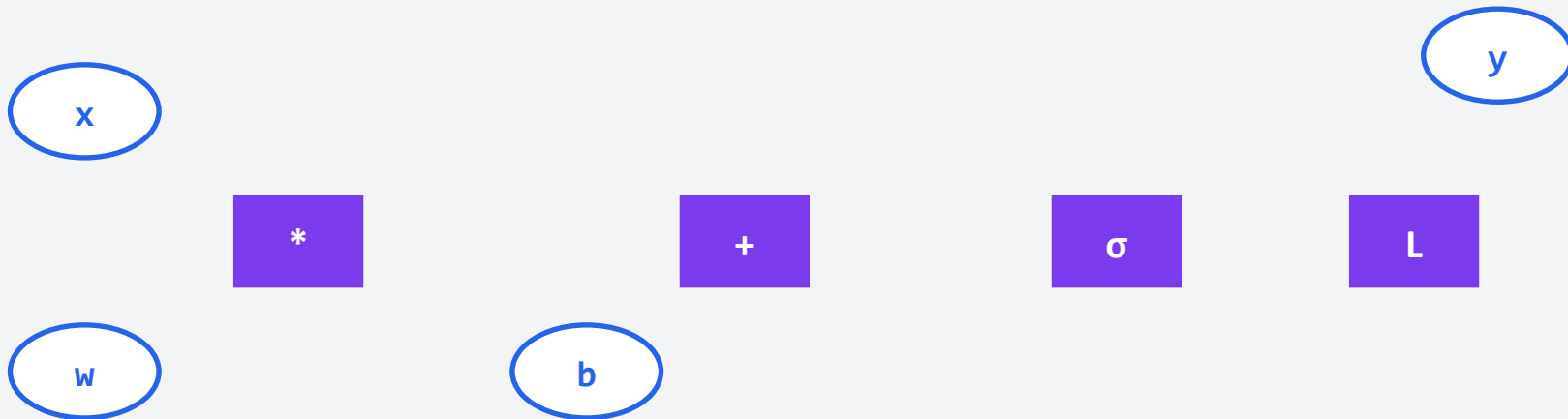
$$dy/dx = f'(g(x)) \cdot g'(x)$$

- Multiplicas derivadas a lo largo de la cadena
- Cadena humana: cada persona modifica y pasa el mensaje

Pausa y piensa

Si hay 10 personas y cada una entiende el 80%, ¿qué fracción llega al final?

Grafo Computacional



Pausa y piensa

Cada operación es un artículo en tu recibo. El total = la pérdida.

Forward vs Backward



Forward

- Calcula salida
- Guarda valores intermedios



Backward

- Calcula gradientes
- Un solo pase

Pausa y piensa

Cocinar = forward. Probar y ajustar la receta = backward.

Flujo del Gradiente



Loss

1



Capa 3

0.25



Capa 2

0.06



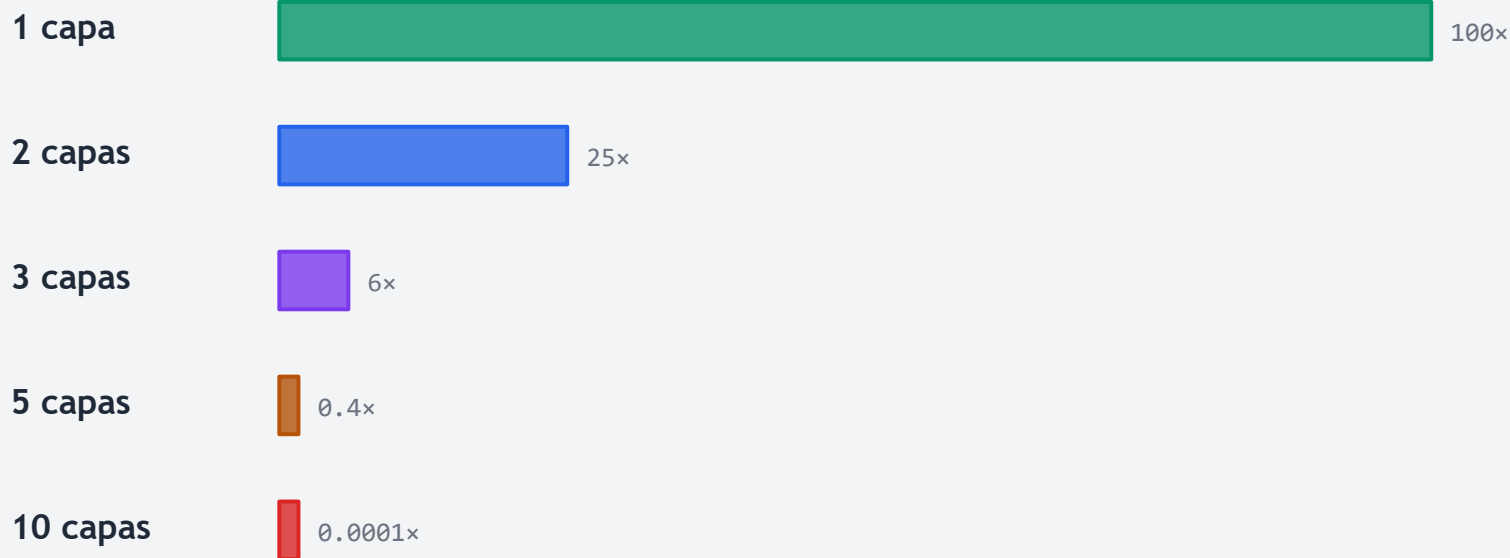
Capa 1

0.015

Pausa y piensa

Como el teléfono descompuesto: el mensaje (gradiente) se debilita en cada capa

Gradientes Vanishing



$\text{sigmoid}'(z) \leq 0.25$ | 10 capas $\rightarrow$ gradiente 0.000001x original

Value Node



Value

data = 5.0

valor numérico

grad = 0.0

gradiente acumulado

_backward()

reparte el error

```
class Value:
    def __init__(self, data, children=(), op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._children = set(children)
        self._op = op
```

Pausa y piensa

Cada número = una ficha de dominó con valor, error acumulado y cómo repartirlo

Add y Mul



SUMA



$$da = 1 \quad db = 1$$

MULTIPLICACIÓN



$$da = b \quad db = a$$

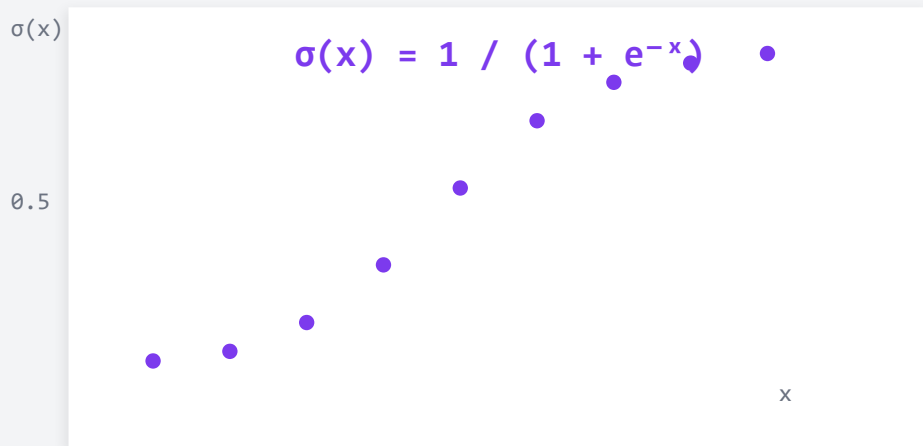
Pausa y piensa

El += es crítico: un Value usado varias veces acumula gradientes de todos los caminos

```
# SUMA: d(a+b)/da = 1  d(a+b)/db = 1
def _backward():
    self.grad += out.grad
    other.grad += out.grad
```

```
# MULT: d(a*b)/da = b  d(a*b)/db = a
def _backward():
    self.grad += other.data * out.grad
    other.grad += self.data * out.grad
```

Sigmoide y Función de Pérdida



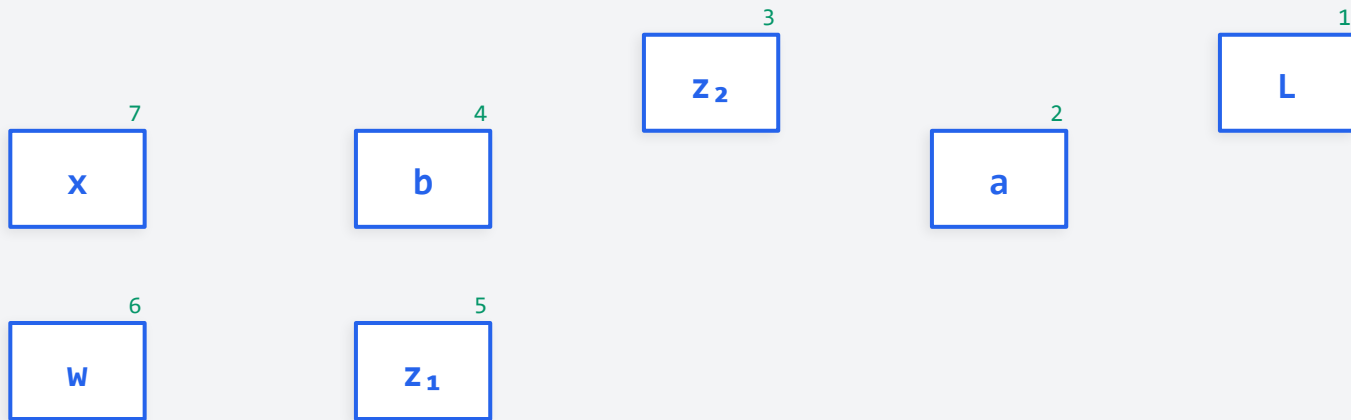
Pausa y piensa

$MSE = (predicho - target)^2$ | Como un termómetro: ¿qué tan lejos

```
def sigmoid(self):
    s = 1/(1 + math.exp(-x))
    def _backward():
        self.grad += s*(1-s) * out.grad
    return Value(s, (self,), 'sigmoid')

def mse_loss(pred, target):
    diff = pred - target
    return diff * diff
```

Backward Pass



Orden de procesamiento inverso: 1 $\rightarrow$ 7

Pausa y piensa

Repartiendo culpas: el jefe (Loss) dice "la culpa es mía = 1" y cada empleado reparte su parte

```
def backward(self):  
    topo = []  
    visited = set()  
    def build_topo(v):  
        if v not in visited:  
            visited.add(v)  
            for child in v.children:  
                build_topo(child)
```

Arquitetura



Neuron

pesos + bias + sigmoide

- $w_1, w_2, \dots$
- bias b
- $\sigma(w \cdot x + b)$



Layer

conjunto de Neuronas

- $n_inputs \rightarrow n_outputs$
- lista de Neuronas



Network

conjunto de Layers

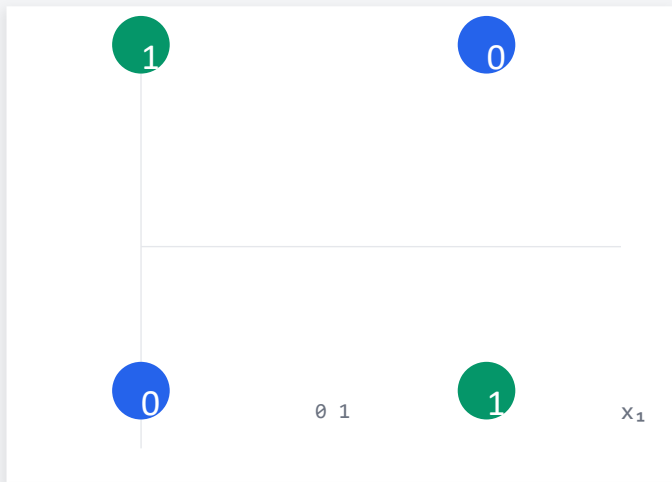
- $sizes = [2, 4, 1]$
- `parameters()`, `zero_grad()`

```
net = Network([2, 4, 1])
net.zero_grad()
total_loss.backward()
for p in net.parameters():
    p.data -= lr * p.grad
```

XOR: Ejemplo Práctico



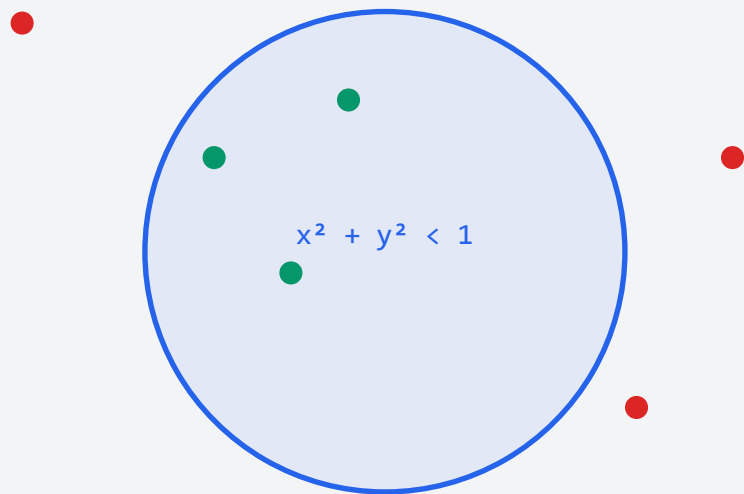
x_2



Input	Pred	Esp
[0,0]	0	0
[0,1]	1	1
[1,0]	1	1
[1,1]	0	0

```
net = Network([2, 4, 1])
for epoch in range(1000):
    total_loss = Value(0.0)
    for inputs, target in xor_data:
        x = [Value(i) for i in inputs]
        loss = mse_loss(net(x), target)
        total_loss += loss
    net.zero_grad()
    total_loss.backward()
    for p in net.parameters():
        p.data -= lr * p.grad
```

Clasificar Círculos



~50%

Accuracy inicial



>95%

Accuracy final

La red descubre el límite circular por sí sola, sin instrucciones explícitas

```
circle_net = Network([2, 8, 1])
for epoch in range(2000):
    random.shuffle(data)
    for inputs, target in data:
        x = [Value(i) for i in inputs]
        pred = circle_net(x)
```

PyTorch



Tu Código

PyTorch

```
total_loss.backward()
```



```
loss.backward()
```

```
p.data -= lr * p.grad
```



```
optimizer.step()
```

```
net.zero_grad()
```



```
optimizer.zero_grad()
```

Mismo algoritmo. Escala industrial. GPUs. Terabytes de datos.

Inferencia vs Entrenamiento



Entrenamiento

Forward + Backward + Update

Los pesos cambian



Inferencia

Forward únicamente

Sin cambios en pesos

Estudiar = entrenamiento. Presentar examen = inferencia.

ChatGPT/Claude usan inferencia: tu prompt fluye forward, ningún peso cambia

Entender backpropagation importa: moldeó cada peso de esa red

Términos Clave



Backpropagation

Asigna culpa (gradientes) a cada peso vía chain rule

Grafo Computacional

Mapa de operaciones: nodos = cálculos

Regla de la Cadena

Multiplicar derivadas a lo largo de la cadena

Gradiente

Cómo cambiaría el error si ajustas un peso

Vanishing Gradient

Gradiente se encoge exponencialmente con sigmoide

Forward Pass

Calcular salida, guardar valores intermedios

Backward Pass

Recorrer el grafo al revés repartiendo gradientes

Learning Rate

Tamaño del ajuste: lr pequeño = lento

Topological Sort

Orden: hijos procesados antes que padres



Lo que aprendimos

- Backprop = 1 backward pass calcula TODOS los gradientes
- Regla de la cadena: multiplicar derivadas hacia atrás
- Value: almacena data, grad y cómo repartir el error
- Suma reparte igual, Multiplicación reparte según el otro valor
- Sigmoides profundos = gradientes que desaparecen
- Construiste el mismo algoritmo que usa PyTorch

Gracias

Próximo: Lesson 04 — ReLU y variantes